

Generative Latent Models for DeepSDF: Gaussian vs. Diffusion in the Small-Data Regime

Amit Benbenishti

Deep Learning for 3D Computer Vision
Hebrew University of Jerusalem

Abstract

DeepSDF (Park et al., 2019) represents a shape category with one MLP and a per-shape latent code, but it does not learn a *distribution* over those codes, so drawing new codes at random rarely decodes to a valid shape. Latent diffusion (3D-LDM (Nam et al., 2022)) fixes this at full ShapeNet scale, but its benefit over a trivial Gaussian prior in the low-data, low-capacity regime is unstudied. We train a DeepSDF auto-decoder, freeze it, and fit three generators over the resulting codes—a multivariate Gaussian, a Gaussian mixture model (GMM), and a DDPM denoising the codes directly—then compare them as the dataset shrinks, $N \in \{10, 50, 150\}$, on ShapeNet chairs (synset 03001627), with latent dimension D held fixed and ablated separately. We evaluate reconstruction (Chamfer, IoU) and generation (Coverage, MMD, 1-NN accuracy). **[TODO: one sentence with the headline generator-comparison result]** Along the way we report two findings that dominated the outcome: raw ShapeNet meshes are not watertight, which silently corrupts the SDF *sign* that DeepSDF trains on, and once that is repaired, added decoder capacity lowers training loss while making the extracted geometry worse. In both cases the training loss improved while the extracted geometry degraded, which is the practical lesson we draw: on a pipeline that ends in Marching Cubes, point-wise loss is a weak proxy for surface quality. Synthetic parametric shapes are used only for pipeline smoke tests.

1 Introduction and Related Work

Implicit shape representations. A signed distance function (SDF) represents a surface implicitly as the zero level set of $f : \mathbb{R}^3 \rightarrow \mathbb{R}$, where $f(\mathbf{x})$ is the signed distance from $\mathbf{x}$ to the surface (negative inside). DeepSDF (Park et al., 2019) learns a *single* MLP $f_\theta(\mathbf{z}, \mathbf{x})$ shared across a shape category, conditioning on a per-shape latent code $\mathbf{z}$; the surface for code $\mathbf{z}$ is $\{\mathbf{x} : f_\theta(\mathbf{z}, \mathbf{x}) = 0\}$, extracted with Marching Cubes (Lorensen and Cline, 1987). It is trained as an *auto-decoder*: the codes are free parameters optimized jointly with θ . Occupancy Networks (Mescheder et al., 2019) and IM-

Net (Chen and Zhang, 2019) are contemporaneous implicit representations, and periodic-activation networks (SIREN (Sitzmann et al., 2020)) improve the fitting of such fields.

Generation over latent codes. DeepSDF’s L2 code regularizer is the MAP estimate under a zero-mean Gaussian prior, so in principle one could sample $\mathbf{z} \sim \mathcal{N}(0, I)$ and decode. In practice the learned codes do not fill an isotropic Gaussian and naive sampling yields broken shapes. 3D-LDM (Nam et al., 2022) instead trains a *latent diffusion model* over DeepSDF codes at full ShapeNet scale, following DDPM (Ho et al., 2020); diffusion also underlies 3D generation via score distillation (DreamFusion (Poole et al., 2023), our HW3). Alternative small-data shape generators include shape VAEs with geometric augmentation (GLASS (Muralikrishnan et al., 2021)) and rigidity-regularized auto-decoders (ARAPReg (Huang et al., 2021)).

Our question and contribution. All of the above operate at scale. We ask a controlled, resource-conscious question: *on a real ShapeNet chair subset with small N and modest D , when does a DDPM over DeepSDF codes beat simpler priors (Gaussian, GMM)?* We keep Stage 1 fixed and vary only the Stage 2 generator as N shrinks on ShapeNetCore chairs, reporting reconstruction and distribution-level generation metrics (Achliopas et al., 2018). A rigorous negative result (simple priors $\approx$ diffusion at small N) is valid and informative.

Our second contribution is diagnostic. Getting *any* coherent chair out of this pipeline turned out to be dominated by two issues that are easy to misattribute to the generator or the optimizer: ShapeNet meshes are not watertight, so the SDF sign used as the training label is largely wrong (Section 2.2); and once repaired, a wider decoder achieved lower training loss while producing *worse* meshes (Sec-

tion 3.3). We document both, since either one silently caps the quality of every prior compared on top of it. All training code is our own (see the code README and src/thirdparty/).

2 Method

2.1 Stage 1: DeepSDF auto-decoder

We assign each of the N training shapes a latent code $\mathbf{z}_i \in \mathbb{R}^D$, initialized from $\mathcal{N}(0, 0.01^2)$, and share a decoder f_θ . The decoder has 8 hidden layers of width 256 with ReLU activations, a skip connection that re-injects the input $[\mathbf{z}; \mathbf{x}]$ at layer 4, weight normalization on the hidden layers, and an unsquashed linear scalar output; dropout is disabled. DeepSDF’s optional tanh output is available in our implementation but left off: the clamped loss of Eq. (1) already restricts supervision to the near-surface band, and the squashed output trained less reliably in our runs. Width 256 rather than the paper’s 512 is a deliberate choice we return to in Section 3.3, where added capacity *hurt* the extracted geometry.

We initialize with *geometric initialization* (Atzmon and Lipman, 2020; Gropp et al., 2020): the output layer starts as $\|\mathbf{x}\| - r$ with $r = 0.5$, so the untrained network is an approximate sphere SDF. Without this the decoder can settle into a degenerate basin where the field is constant in space, never crosses zero, and yields no extractable surface.

Sampling supervision. For each normalized mesh (centered, scaled into the unit sphere) we sample points with 90% near the surface (surface points perturbed by two Gaussians, $\sigma \in \{0.005, 0.02\}$) and 10% uniformly in $[-1, 1]^3$, and record their signed distances.

Loss. Following DeepSDF we minimize a clamped- L_1 reconstruction loss plus code regularization:

$$\mathcal{L} = \frac{1}{|B|} \sum_{(i, \mathbf{x}) \in B} |\hat{f}_\theta(\mathbf{z}_i, \mathbf{x}) - \hat{s}(\mathbf{x})| + \lambda \frac{1}{|B|} \sum_i \|\mathbf{z}_i\|_2^2, \quad (1)$$

where $s(\mathbf{x})$ is the ground-truth SDF, $\hat{a} \equiv \text{clamp}(a, \delta) = \min(\delta, \max(-\delta, a))$ with $\delta = 0.1$, and $\lambda = 10^{-4}$. Clamping concentrates capacity on the near-surface band that Marching Cubes actually reads, rather than on far-field distances. The decoder weights θ and the codes $\{\mathbf{z}_i\}$ are optimized jointly with Adam.

2.2 Data preparation: making the SDF sign meaningful

The label in Eq. (1) is a *signed* distance, so it presupposes a well-defined inside and outside — that is, a closed surface. Raw ShapeNetCore OBJs do not satisfy this. They are assembled *shells* rather than solids: a representative chair from synset 03001627 decomposes into 313 disconnected components (seat, each leg, back, cushions) that are never welded into one manifold. On such input, trimesh’s winding-number signed-distance query has no coherent interior to test against and its sign becomes close to meaningless. Measured on one chair, only $\approx 1\%$ of sampled points were labelled inside; after the repair below, the *same* sample locations gave $\approx 38\%$.

This is a training-*data* bug, and it has a specific failure signature. With nearly every near-surface label positive, the network is asked to fit something closer to an unsigned distance field with almost no interior, so its zero level set is not a closed surface and Marching Cubes returns disconnected fragments. Meanwhile the point-wise loss stays low — the decoder faithfully fits the labels it was given — so the monitored training metric never reveals the problem. No amount of decoder or optimizer tuning downstream can repair a corrupted label.

Repair. We voxelize the normalized mesh at pitch $1/64$, fill the interior, and re-mesh the filled occupancy grid with Marching Cubes (make_watertight in src/data/normalize.py). This yields a single closed manifold solid regardless of how broken the input topology was; we re-normalize afterwards because voxelization shifts the bounding box slightly. The repair is applied per mesh only when trimesh reports the input as non-watertight, and is covered by a unit test on a deliberately fragmented mesh.

The trade-off is that detail thinner than the pitch is lost — thin slats and spindles thicken or merge. We accept it deliberately: a slightly thickened chair with correct signs is a learnable target, whereas a perfectly detailed chair with arbitrary signs is not. The cost is preprocessing time, which dominates the data build at $N = 150$.

2.3 Stage 2: generative priors over frozen codes

We freeze f_θ and $\{\mathbf{z}_i\}$ and fit three generators.

Gaussian (minimal). We fit a multivariate Gaussian $\mathcal{N}(\mu, \Sigma)$ to the codes by maximum likelihood (full covariance with a small ridge) and sample $\mathbf{z} = \mu + L\epsilon$ with $\Sigma = LL^\top$.

GMM (intermediate). We fit a full-covariance Gaussian mixture with K components via EM (implemented from scratch). With small N , K is capped so each component receives enough codes; this prior sits between a single Gaussian and diffusion on the minimal-to-expressive spectrum.

Latent DDPM (expressive). We train a denoising diffusion model directly on the D -dimensional codes, reusing the noise-prediction objective of HW3 (Ho et al., 2020). With a cosine schedule (Nichol and Dhariwal, 2021) $\{\bar{\alpha}_t\}$, the forward process is $\mathbf{z}_t = \sqrt{\bar{\alpha}_t}\mathbf{z}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$, and a small MLP $\epsilon_\phi(\mathbf{z}_t, t)$ (sinusoidal time embedding, residual blocks) is trained with $\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t,\epsilon} \|\epsilon_\phi(\mathbf{z}_t, t) - \epsilon\|_2^2$. We generate by ancestral sampling from $\mathcal{N}(0, I)$.

2.4 Inference and meshing

To realize a sampled code we evaluate $f_\theta(\mathbf{z}, \cdot)$ on a dense grid over $[-1, 1]^3$ and run Marching Cubes at level 0 to obtain a mesh, from which we sample a surface point cloud for the metrics.

3 Evaluation and Results

3.1 Setup

All experiments use ShapeNetCore chairs (synset 03001627).

Experimental design. The proposal specified a full $N \times D$ grid. We instead run the N -sweep at a *fixed* D and treat D as a single ablation. The reason is an earlier observation, made before the mesh repair of Section 2.2 was in place: at $N = 50$, raising D from 16 to 32 *degraded* reconstruction Chamfer rather than improving it, echoing the decoder-width result of Section 3.3. Since both capacity axes pointed the same way, crossing D with every N would largely re-measure one effect three times. The main sweep is therefore $N \in \{10, 50, 150\}$ at $D = 16$ (three cells, each with all three generators), plus one $D = 32$ cell at $N = 50$ re-measured under the repaired pipeline (Section 3.5) so the ablation is comparable to the rest of the results. This keeps the comparison the paper is actually about — Gaussian vs. GMM vs. DDPM as N shrinks — free of a confounded capacity axis.

Training. Each cell trains Stage 1 for 8000 iterations with Adam (learning rate 10^{-3} for both decoder and codes), sampling 48 shapes and 1024 points per shape per iteration, then fits the three Stage-2 generators on the frozen codes. The GMM uses $K = 5$ components (capped so no component receives fewer than 5 codes). The DDPM uses 200 timesteps with a cosine schedule (Nichol and Dhariwal, 2021), a 4-layer width-256 MLP with a 64-d sinusoidal time embedding, trained 3000 iterations at learning rate 10^{-3} .

Data and evaluation. Meshes are normalized to the unit sphere and repaired (Section 2.2); SDF supervision uses 8000 points per shape (90% near-surface, 10% uniform). Reconstruction is measured on up to 10 training shapes per cell at Marching-Cubes resolution 48^3 (IoU at 32^3). For generation we draw 40 samples per generator and compare against 40 held-out chairs, disjoint from training by construction (offset by N in the directory listing). Hyper-parameters live in `configs/shapenet_grid.yaml`; `scripts/run_grid.py` reproduces every cell on a single Colab GPU (T4/A100) and `scripts/make_figures.py` regenerates every table and figure in this section from the resulting `summary.json`.

3.2 Metrics

Reconstruction: Chamfer distance and volumetric IoU between each decoded training code and its ground truth. **Generation** (Achlioptas et al., 2018), computed between a set of generated shapes and a held-out reference set of real shapes, using Chamfer distance between surface point clouds: *Coverage* (fraction of references matched, measures diversity/mode coverage), *MMD* (minimum matching distance, measures fidelity), and *1-NN accuracy* (a two-sample test; 0.5 is ideal, indicating the generated and real distributions are indistinguishable).

3.3 Failure modes and what they diagnosed

Most of the effort in this project went into obtaining *any* coherent geometry, not into the generator comparison. We report the failures because each one is easy to misattribute, and two of them are only visible if you look at the extracted surface rather than the loss.

Latent-code collapse. In several configurations the mean code norm decayed toward zero (e.g.

from 0.036 to 0.0005 within 2000 iterations), after which the decoder ignores $\mathbf{z}$ and emits one average shape. The cause is an interaction between two individually sensible choices. Geometric initialization deliberately zeroes the latent columns of the first layer so the initial field is a code-independent sphere — but then $\partial\mathcal{L}_{\text{recon}}/\partial\mathbf{z}$ is *exactly* zero at step 1, and the only gradient reaching the codes is the regularizer $\lambda\|\mathbf{z}\|^2$ pulling them to the origin. Training becomes a race between the codes bootstrapping a nonzero coupling and the regularizer erasing them.

The instructive part is that simply enlarging that initialization was not enough: raising the latent-column standard deviation $100\times$ (from 10^{-4} to 10^{-2}) still collapsed at the *same rate* at width 512. Adam normalizes each parameter’s step by its own gradient RMS, so it moves in whichever direction is most *consistent* over time, largely irrespective of raw gradient magnitude. The regularizer supplies a tiny but perfectly consistent pull toward zero at every step, which a weak and noisy reconstruction signal loses to even at comparable magnitude. What fixed it was removing the competing force during the critical window: a linear warmup ramping λ from 0 over the first iterations (code_reg_warmup_iters), together with the larger latent-column init.

Optimizer sensitivity to width. At width 512, a learning rate of 10^{-3} for both decoder and codes stalled training outright — the reconstruction loss stayed flat near 0.09 while $\|\mathbf{z}\|$ collapsed. Reducing to 3×10^{-4} restored normal training. Wider weight-normalized layers simply did not tolerate the learning rate that worked at width 256.

Eikonal regularization: a scale-mismatch trap. We also tried adding the unit-gradient penalty $(\|\nabla_{\mathbf{x}}f\| - 1)^2$ used by IGR (Gropp et al., 2020). At the literature default weight of 0.1 it destroyed training: the clamped- L_1 data term is $O(\delta) = O(10^{-2})$, whereas the eikonal term is unclamped and $O(1)$, so the regularizer dominated the objective by roughly an order of magnitude and drove the field to a degenerate, nearly-code-independent solution with no zero crossing inside the evaluation cube. The default assumes an unclamped data loss of comparable scale; with clamping, the weight must be scaled down by roughly δ . We ultimately trained without it.

The capacity paradox, and what it revealed.

Widening the decoder from 256 to 512 reduced the training loss yet produced *worse* Chamfer, worse IoU, and visibly worse meshes. That combination is diagnostic rather than merely disappointing: a model that fits its labels better while producing worse geometry implies the labels disagree with the geometry. This is what sent us to audit the supervision itself, where the sign turned out to be wrong for roughly 99% of sampled points (Section 2.2). We note the ordering honestly — capacity and optimizer changes were attempted first and did not help — because the *pattern* of failure, not any single metric, is what identified the cause. It also explains why we keep width at 256: until the label bug was fixed, extra capacity only fit the wrong target more precisely.

3.4 Quantitative results

Table 1 reports every metric. Figure 1 shows how generation quality changes with N for each generator at $D = 16$, and Fig. 2 the corresponding reconstruction trend. Training curves are in Fig. 3 and qualitative samples in Fig. 4.

3.5 Ablation: latent dimension

The final block of Table 1 reports the $D = 32$ cell at $N = 50$, to be compared against the $D = 16$ cell at the same N . Doubling the latent dimension doubles the per-shape capacity available to Stage 1 while leaving the decoder and the training budget unchanged, so it is the cheapest available test of whether the codes — rather than the decoder — were the bottleneck. It also determines whether a D -sweep is worth spending the remaining compute on: if $D = 32$ does not help at the middle N , crossing D with every N mostly re-measures the same effect.

3.6 Discussion

How much of the budget reaches the surface.

Two quantities have to be read together in Table 1. Reconstruction Chamfer/IoU bound what *any* prior can achieve, because a sampled code is decoded by the same network: if the training codes themselves do not decode cleanly, no prior over those codes can. The valid ratio then says how often a sampled code produces an extractable surface at all. A generator can look strong on Coverage and MMD while quietly failing on a large fraction of its samples, since those metrics are computed only over samples that yielded a mesh.

Table 1: Reconstruction and generation metrics. Recon-CD and MMD are lower-is-better; IoU and Coverage higher-is-better; 1-NN accuracy is best near 0.5. Reconstruction is a property of Stage 1 and is therefore shared by the three generators within a cell.

N	D	Gen.	Recon-CD	IoU	Coverage	MMD	1-NN	Valid
[TODO: awaiting the grid run — regenerate with <code>make_figures.py</code>]								

PLACEHOLDER

Generation metrics vs. N

Awaiting the ShapeNet grid run. Regenerate with:
`python scripts/make_figures.py --input outputs/shapenet_scoped --figures ../report/figures`

Figure 1: Generation metrics vs. number of training shapes N , per generator, at the fixed latent dimension $D = 16$. The dashed line marks the ideal 1-NN accuracy of 0.5.

We therefore treat valid ratio as a first-class result rather than a diagnostic. **[TODO: state reconstruction CD/IoU and the valid-ratio range actually observed]**

Do the priors differ at all? This is the paper’s question, and the informative outcome may well be a negative one. At small N the empirical code distribution is a handful of points in $\mathbb{R}^{16}$; a full-covariance Gaussian already has $D + D(D+1)/2 = 152$ parameters to fit N vectors, so the mixture and the diffusion model are being asked to find structure in a sample that may not support it. If the three priors are within noise of each other, the practical conclusion is that diffusion’s extra machinery buys nothing in this

regime — which is precisely the claim the proposal set out to test, and it inverts the usual reading of 3D-LDM’s result by showing where its advantage comes from (scale) rather than disputing it. **[TODO: compare the three generators on Coverage/MMD/1-NN; say whether the gaps exceed the spread across N , and note any consistent ordering]**

The 1-NN sanity check. 1-NN accuracy is the strictest of the three generation metrics: 0.5 means a classifier cannot tell generated shapes from real ones, and values near 1.0 mean it always can. Under a budget of 40 samples against 40 references, and with reconstructions that are themselves imperfect, we expect values far above 0.5 — a re-

PLACEHOLDER

Reconstruction vs. N

Awaiting the ShapeNet grid run. Regenerate with:
`python scripts/make_figures.py --input outputs/shapenet_scoped --figures ../report/figures`

Figure 2: Reconstruction Chamfer distance and IoU vs. N at $D = 16$.

PLACEHOLDER

Stage-1 and DDPM training losses

Awaiting the ShapeNet grid run. Regenerate with:
`python scripts/make_figures.py --input outputs/shapenet_scoped --figures ../report/figures`

Figure 3: Stage-1 clamped- L_1 reconstruction loss and Stage-2 DDPM noise-prediction loss for each cell.

minder that Coverage and MMD alone can flatter a generator whose samples are individually implausible. **[TODO: report the 1-NN range and whether any generator approaches 0.5]**

Trend with N . More shapes cut two ways here. The prior sees a larger sample, which should help it, but Stage 1 must fit more geometry with a fixed decoder and a fixed iteration budget, which should hurt reconstruction — and reconstruction quality caps everything downstream. Figure 1 and Fig. 2 separate these effects. **[TODO: state which effect dominates between $N = 10$ and $N = 150$]**

Latent dimension. Section 3.5 compares $D = 32$ against $D = 16$ at $N = 50$ under the repaired pipeline. **[TODO: report whether $D = 32$ helps, hurts, or is neutral, and reconcile with the pre-repair observation that motivated fixing $D = 16$]**

Qualitative gallery. Figure 4 is the check that the metrics cannot provide. Chamfer distance between surface point clouds is fairly forgiving of a shape that has roughly the right mass in roughly the right places, so a recognizable chair and a plausible-but-wrong blob can score similarly. Reading the gallery alongside the table is what caught the label bug of Section 2.2 in the first place: the numbers were unremarkable while the meshes were fragments. Failed decodes are drawn as an explicit *fail* marker rather than silently omitted, so the figure and the valid ratio agree. **[TODO: describe what the samples actually look like per prior — coherence, diversity, mode collapse, and any visible difference between GMM and DDPM]**

PLACEHOLDER

GT / reconstruction / prior samples

Awaiting the ShapeNet grid run. Regenerate with:
`python scripts/make_figures.py --input outputs/shapenet_scoped --figures ../report/figures`

Figure 4: Qualitative ShapeNet results, rendered as shaded solids. Rows: ground-truth chair, reconstruction from its learned code, then samples from the Gaussian, GMM, and DDPM priors.

4 Conclusion

We compared Gaussian, GMM, and DDPM priors over frozen DeepSDF codes on a ShapeNet chair subset as the dataset shrank, $N \in \{10, 50, 150\}$ at $D = 16$, with $D = 32$ ablated separately. The goal was not state of the art but a controlled answer to when diffusion is worth its complexity on real meshes under a modest compute budget. **[TODO: answer the question in one or two sentences, with the numbers that support it]**

The most transferable lesson is not about the priors at all. Two failures dominated our results, and both presented as *optimization* problems while

actually being something else: non-watertight input meshes corrupting the SDF sign that DeepSDF trains on (Section 2.2), and added capacity simultaneously lowering training loss and worsening extracted geometry (Section 3.3). In both cases the training loss — the quantity one naturally monitors — moved in the reassuring direction while the artifact we cared about got worse. On a pipeline that ends in Marching Cubes, point-wise loss is a weak proxy for the surface, and geometry has to be inspected directly.

Limitations include the cost of mesh repair, the sensitivity of Marching Cubes to under-trained fields, the single synset, and the modest sample

counts behind the generation metrics (40 samples against 40 references), which limits how finely Coverage and 1-NN accuracy can be resolved. Future work could scale N within the chairs category, retune the optimizer for wider decoders instead of fixing width, and add flow-based priors as a fourth point on the minimal-to-expressive spectrum.

References

- Panos Achlioptas, Olga Diamanti, Ioannis Mitliagkas, and Leonidas Guibas. 2018. Learning representations and generative models for 3d point clouds. In *International Conference on Machine Learning (ICML)*.
- Matan Atzmon and Yaron Lipman. 2020. SAL: Sign agnostic learning of shapes from raw data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Zhiqin Chen and Hao Zhang. 2019. Learning implicit fields for generative shape modeling. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Amos Gropp, Lior Yariv, Niv Haim, Matan Atzmon, and Yaron Lipman. 2020. Implicit geometric regularization for learning shapes. In *International Conference on Machine Learning (ICML)*.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. 2020. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Bo Huang, Zhiqian Yi, Zherong Yang, Danny M. Kaufman, Daniel Cohen-Or, and Qixing Huang. 2021. ARAPReg: An as-rigid-as-possible regularization loss for learning deformable shape generators. In *IEEE/CVF International Conference on Computer Vision (ICCV)*.
- William E. Lorensen and Harvey E. Cline. 1987. Marching cubes: A high resolution 3d surface construction algorithm. In *ACM SIGGRAPH Computer Graphics*.
- Lars Mescheder, Michael Oechsle, Michael Niemeyer, Sebastian Nowozin, and Andreas Geiger. 2019. Occupancy networks: Learning 3d reconstruction in function space. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Sanjeev Muralikrishnan, Siddhartha Chaudhuri, Noam Aigerman, Vladimir Kim, Matthew Fisher, and Niloy Mitra. 2021. GLASS: Geometric latent augmentation for shape spaces. *arXiv preprint arXiv:2108.03225*.
- Gimin Nam, Mariem Khelifi, Andrew Rodriguez, Alberto Tono, Linqi Zhou, and Paul Guerrero. 2022. 3D-LDM: Neural implicit 3d shape generation with latent diffusion models. *arXiv preprint arXiv:2212.00842*.
- Alexander Quinn Nichol and Pratul D. Dhariwal. 2021. Improved denoising diffusion probabilistic models. In *International Conference on Machine Learning (ICML)*.
- Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. 2019. DeepSDF: Learning continuous signed distance functions for shape representation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. 2023. DreamFusion: Text-to-3d using 2d diffusion. In *International Conference on Learning Representations (ICLR)*.
- Vincent Sitzmann, Julien N. P. Martel, Alexander W. Bergman, David B. Lindell, and Gordon Wetzstein. 2020. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems (NeurIPS)*.

A Original Proposal

The approved project proposal is reproduced below.

Title. Generative Latent Models for DeepSDF: Gaussian vs Diffusion in the Small-Data Regime. *Author:* Amit Benbenishti.

Problem definition. DeepSDF learns one latent code per shape, but does not learn a distribution over codes—random codes decode to invalid shapes. I train a generative model over the latent space (not over shapes directly) and study which simple prior works when data and capacity are limited.

Approach. *Stage 1:* train a DeepSDF auto-decoder on a shape collection; collect latent codes $\{z_i\}$. *Stage 2 (frozen decoder):* fit two generators on $\{z_i\}$ —(1) Gaussian (mean + covariance), (2) DDPM diffusion over z (same noise-prediction setup as HW3, applied to vectors). *Inference:* sample z from a generator $\rightarrow$ decode with $f(z, x) \rightarrow \text{SDF} \rightarrow \text{mesh}$. Sweep $N \in \{10, 50, 150\} \times D \in \{16, 32\}$ and compare generators. Goal is not SOTA—it is to identify when diffusion is worth the extra complexity over a Gaussian. All training code is my own.

Motivation. 3D-LDM uses latent diffusion over DeepSDF codes at full ShapeNet scale; naive Gaussian sampling from the same latent space often fails. I study the minimal-resource regime (small N , small D , Colab) where that tradeoff is unknown.

Validation. *Data:* ShapeNet subset (~ 50 – 200 meshes) or synthetic shapes + stanford_bunny.obj. *Metrics:* Chamfer + IoU (reconstruction); Coverage, MMD, 1-NN accuracy (generation); qualitative samples and interpolations. *Mid-point checks:* (1) overfit one shape, (2) train DeepSDF on ~ 10 shapes and extract codes, (3) sample one mesh from Gaussian and one from diffusion. *Final:* full $N \times D$ grid + generator comparison + degradation curves, including negative results (e.g., Gaussian matches diffusion at small N).

Related work. DeepSDF (Park et al., 2019)—auto-decoder and latent codes (Stage 1). 3D-LDM (Nam et al., 2022)—latent diffusion over DeepSDF codes at scale; does not study small N/D or Gaussian vs diffusion explicitly. DreamFusion / DDPM (Poole et al., 2023; Ho et al., 2020)—diffusion training (HW3 connection).